FPGA Network and Specification Document

Project Name: tinyODIN-Olla

OLLA-FPGAN-001, Rev. A

Version 0.1

Prepared By: H. Basri Bilge, FPGA Engineer Intern

Issued Date

November 3, 2025

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

Figure 1 Revision History

Rev No.	*Date*	*Description*	*Author*
v0.1	03/11/2025	Initial Draft Creation	H. Basri Bilge

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

1. Project Overview and Scope

1.1 Purpose and Objectives

The primary purpose of this FPGA design is to create a **dedicated test environment and stimulus generator** for the **Olla neuromorphic chip**, facilitating the functional validation of its core operations.

This design serves as a robust platform to interface with Olla, allowing for the comprehensive verification of a pre-designed Spiking Neural Network (SNN) based on Charlotte Frenkel's work.

Key Objectives:

The FPGA design must fulfill the following specific, measurable, achievable, and time-bound (SMART) objectives:

1. Functional Validation: Successfully perform functional validation of the Olla chip by implementing the SNN processor logic derived from the **tinyODIN** architecture (crossbar array).
2. Interface Adaptation: Implement necessary modifications (major and minor) to the existing tinyODIN-related modules to ensure seamless and correct communication with the proprietary **Olla chip interface**.
3. Protocol Implementation: Successfully implement and utilize the **asynchronous 4-phase dual-rail handshake protocol** to send stimuli to the Olla chip and reliably receive and interpret the expected neural output data.
4. Test Platform Creation: Deliver a stable and reusable FPGA configuration that serves as a **baseline test harness** for future validation and characterization efforts of the Olla chip.

1.2 Scope and Deliverables

This subsection defines the precise boundaries of the FPGA design effort, clarifying which components and functionalities are included in the work and which are specifically excluded.

1.2.1 Inclusions

The scope of this FPGA design project specifically **includes** the following core components and tasks:

1. Controller (controller.v):
    - Scope: **Significant modifications** were implemented to handle comprehensive test sequencing. Changes involve the sequential and combinational logic, including the addition of **critical Finite State Machine (FSM) states** required to govern the test execution flow.
2. Synaptic Core (synaptic_core.v):
    - Scope: The module's role underwent a **critical functional change** to manage and hold the data for the **first layer of cascaded memory**. The module is now responsible for referencing memory data information stored within the associated BRAM_Lut block.
3. Neuron Core (neuron_core.v):
    - Scope: **Major changes** were applied to the core neuron processing logic. This includes its sub-modules (lif_neuron.v, BRAM_256x32_wrapper, BRAM_Synapse) to integrate Olla-specific functionalities and ensure the correct neural firing behaviour within the test environment. A **second FSM** was specifically created within this module to handle the handshake protocol with the Olla chip and process both neural and time reference events.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

4. AER Output (aer_out.v):
    - Scope: **Minor changes** were made to maintain the functionality of the Address Event Representation (AER) output logic. Necessary adjustments were implemented to ensure proper operation relative to the significant changes made in the **Neuron Core** module.
5. LED Visualization Modules (olla_to_LED.v / aer_to_LED.v):
    - Scope: Development and full implementation of dedicated indicator logic. These modules provide real-time visual feedback (via LEDs) on the test board, specifically showing activity corresponding to the Olla response (olla_to_LED.v) and the generated stimulus (aer_to_LED.v).
6. Scheduler (scheduler.v):
    - Scope: Only **one specific modification** was introduced into this module to correctly process internal events necessary for managing the **second layer** of the Spiking Neural Network (SNN).

1.2.2 Exclusions

SPI module's function is **disabled** and **not utilized** within the current design architecture. It is reserved for future applications and remains outside the scope of validation and development effort.

1.3 Hardware Platform and Tools

This subsection details the specific hardware platform used for implementation and the Electronic Design Automation (EDA) tools utilized for the synthesis, place-and-route, and simulation of the FPGA design.

1.3.1 Target Hardware Platform

The design is targeted for implementation on a high-performance FPGA development kit to meet the strict timing and resource requirements of the neuromorphic interface.

Figure 2 Target Hardware

Component	***Specification / Model***	***Purpose in Project***
FPGA Device	Xilinx Kria KR260 UltraScale+ MPSoC	Provides high-density logic resources and high-speed I/O pins necessary for complex SNN adaptation logic and high-frequency communication with the Olla chip.
Development Board	Xilinx Kria KR260 UltraScale+ MPSoC Carrier Board	Serves as the primary test harness, offering robust power delivery and the necessary PMOD connector interface for the Olla chip interposer.
External Interface	Probe Station with 100 needles probe card	Connects the FPGA's high-speed I/O pins directly to the Olla chip, enabling the asynchronous 4-phase dual-rail protocol communication.
System Clock	100MHz On-board Oscillator	The primary system clock source, from which all internal clocks (e.g., for controller.v) are derived using MMCM/PLL resources.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

1.3.2 Electronic Design Automation (EDA) Tools

The following software versions were used to develop, synthesize, and verify the HDL code and generate the final bitstream.

Figure 3 EDA Tools

Tool Category	*Software / Version*	*Purpose*
Synthesis & P&R	**Xilinx Vivado** Design Suite **2024.2**	Used for all stages of implementation, including logic synthesis, physical placement and routing, timing analysis, and bitstream generation.
Simulation	**Vivado Simulation 2024.1**	Used for pre-synthesis and post-synthesis functional verification of all modified HDL modules and the top-level test harness.
Code Editor	**Visual Studio Code** with Verilog/VHDL extensions	Primary environment for HDL code creation and maintenance.
Version Control	**Git**	Used for source code management, change tracking, and collaboration for all HDL and documentation files.

2. Requirements and Specifications

This section defines all necessary functions, performance metrics, and constraints that the FPGA design must meet to successfully serve as the Olla chip validation platform.

2.1 Functional Requirements

Functional requirements define **what** the system must do from a behavioral perspective. Each requirement should be uniquely identifiable for traceability.

Figure 4 Functional Requirements

ID	*Description*	*Source / Rationale*	*Related Module(s)*
FR-001	**Olla Handshake Protocol:** The FPGA shall correctly implement the **asynchronous 4-phase dual-rail handshake protocol** to ensure robust and reliable communication with the Olla chip for both read and write operations.	Core project requirement; essential for physical interface.	neuron_core.v
FR-002	**Stimulus Generation:** The FPGA shall generate and inject pre-defined spike patterns (stimuli) into the Olla chip based on an internal memory structure, controlled by the controller FSM.	Test sequencing requirement; derived from Charlotte Frenkel's SNN structure.	aer_stimulus_generator

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

FR-003	**Neural Response Capture:** The FPGA shall capture and decode the resulting neural spike events and time reference events transmitted asynchronously from the Olla chip's output.	Core project objective (Validation).	neuron_core.v (Second FSM)
FR-004	**Memory Adaptation:** The adapted Synaptic Core logic shall correctly manage and access memory data (BRAM_Lut) necessary for the cascaded memory's first layer, reflecting the changes in its role.	Derived from synaptic_core.v modifications.	synaptic_core.v
FR-005	**Internal Event Processing:** The Scheduler module shall process internal events specifically for the second layer of the SNN, as per the single change made in scheduler.v.	Derived from scheduler.v modification.	scheduler.v
FR-006	**Visual Feedback:** The FPGA shall drive dedicated on-board LEDs to provide immediate visual confirmation of key activities, including stimulus transmission and successful Olla response capture.	Debugging and operational status monitoring.	olla_to_LED.v, aer_to_LED.v

2.2 Interface Requirements

This section defines the electrical, physical, and protocol specifications necessary for the FPGA design to communicate reliably with external components, particularly the Olla neuromorphic chip.

2.2.1 FPGA to Olla Chip Interface

The primary interface involves low-level asynchronous communication using the proprietary dual-rail protocol.

Figure 5 Interface Requirements

ID	*Description*	*Specification*	*Rationale*
IR-001	**I/O Standard and Voltage**	All dedicated I/O pins interfacing with the Olla chip shall use the **1.8V LVCMOS** (Low-Voltage Complementary Metal-Oxide Semiconductor) standard. However, **PMOD** connecters have **3.3V LVCMOS** standard. **Voltage level shifter is mandatory.**	Required to match the electrical characteristics and power constraints of the Olla chip.
IR-002	**Asynchronous Protocol Implementation**	The logic must support the **4-phase dual-rail handshake protocol** using **unidirectional** request/acknowledge (Req/Ack) signal pairs for both data transmission and reception.	Core requirement for robust, clockless communication with the neuromorphic chip.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

IR-003	**Pin Assignment**	The I/O pins used for the Olla interface shall be mapped to a specific bank on the FPGA to ensure signal integrity and proper alignment with the interposer board.	Critical for minimizing skew and signal degradation on the high-speed link.
IR-004	**Olla Data/Event Channels**	The interface shall provide distinct input channels for receiving **Neural Events** and **Time Reference Events** from the Olla chip.	Necessary for the Neuron Core FSM to process different types of output data correctly.

3. Architecture and Block Diagrams

This section provides a top-level view of the FPGA design, detailing the main functional blocks, their interfaces, and the overall data and control flow within the Olla validation platform.

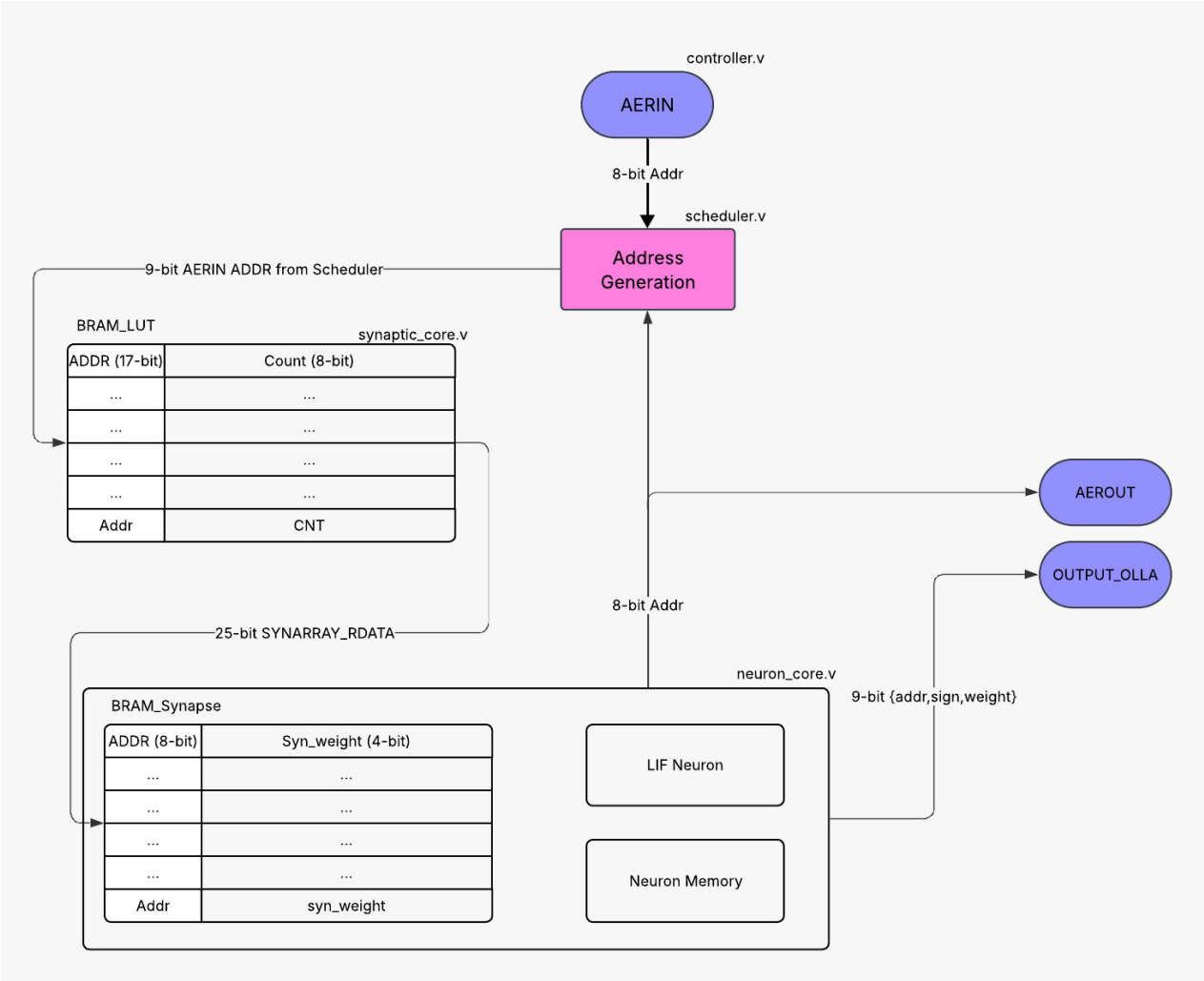
3.1 Top-Level Architecture

The Olla Validation Platform design is primarily structured around the modified tinyODIN core, which acts as the **Stimulus and Response Manager**, integrated with custom I/O and control logic.

- **Central Role:** The FPGA acts as the **master device**, governing the test sequence, generating stimuli, and interpreting the asynchronous response from the Olla chip.
- **Key Data Flow:** The general flow involves the Controller initiating a test sequence, feeding spike patterns via the Stimulus Generator, communicating with Olla via the dual-rail protocol, and finally capturing the output through the Neuron Core logic.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

Figure 6 Block Diagram of Network



Main Functional Blocks:

1. **Controller Core (controller.v and neuron_core.v):** The primary and secondary FSM responsible for system orchestration. They manages the test sequence, initializes the memory blocks, and controls the flow of data between the stimulus generator and the dual-rail protocol engine.

2. **tinyODIN SNN Core (Modified):** Contains the Synaptic Core and Neuron Core logic. These blocks are repurposed to manage the SNN data structure and process the I/O handshake, integrating the specialized second FSM for Olla communication.

3. **AER I/O Manager (neuron_core.v):** Handles the external data formatting for debug and ensures that captured Olla responses are correctly buffered.

4. Design Details

This section provides a detailed explanation of the internal logic of the critical modules, focusing on algorithms, control mechanisms, and memory structures implemented in the HDL code.

4.1 Finite State Machine (FSM) Diagrams

Since the Controller and the modified Neuron Core rely heavily on state machines for sequencing and protocol handling, detailing their FSMs is essential for understanding the design flow.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

- **FR-001** and **FR-003** requirements are fundamentally governed by these FSMs.

4.1.1 Controller FSM

The Finite State Machine (FSM) serving as the Test Sequencer is responsible for managing the high-level operational flow of the FPGA validation platform. Its core function is to oversee the handling of spike traffic, both from the internal scheduler (stimulus) and the external Address-Event Representation (AER) input, which corresponds to Olla responses.

Initial/Wait State: WAIT

The FSM begins operation in the WAIT state, where the system remains idle. This state persists as long as the AER Output controller (AEROUT_CTRL_BUSY) is engaged in data transmission. No further actions are taken until the output controller becomes available.

Transitioning Out of WAIT

The FSM transitions out of the WAIT state based on specific triggers:

- Scheduler Full Condition (SCHED_FULL): If the internal Scheduler is full, the FSM prioritizes processing the scheduled events. The type of event determines the transition:
- If the event is a Virtual Spike (|SCHED_DATA_OUT[12:9]), the FSM advances to POP_VIRT.
- If the event is a Neuron Spike, the FSM advances to POP_NEUR.

External Event Request (AERIN_REQ_sync): If an external event request is detected from the Olla chip, the FSM evaluates the event type:

If the event is a Time Reference event (tref_event), the FSM transitions to TREF.

If the event is a spike event (virt_event or neuron_event), the FSM transitions to PUSH.

Scheduler Not Empty (~SCHED_EMPTY): If the Scheduler contains events, the FSM proceeds to pop the events according to the same logic as above.

AER Input Handling

- TREF → TREF_WAIT: This transition state is used to process Time Reference events.
- PUSH → WAIT_REQDN: Upon receiving a spike event, the FSM acknowledges the event and waits until the Request signal is de-asserted before proceeding.

AER Output Handling

- POP_NEUR → POP_NEUR_WAIT: The FSM prepares to send a neuron event.
- POP_NEUR_WAIT → AER_POP: This transition occurs when the Neuron Core has finished processing (i.e., !NEUR_CTRL_BUSY).
- AER_POP → WAIT: The FSM remains in this state until the AER Output controller completes transmission (!AEROUT_CTRL_BUSY), after which it returns to the main idle state.

Cleanup States

The states WAIT_SPIDN and WAIT_REQDN serve as cleanup phases. They ensure that handshake signals or programming events are fully de-asserted before the FSM re-enters the main WAIT loop, maintaining protocol integrity and preventing unintended state transitions.

4.1.2 Neuron Core SNN Processing FSM

This FSM handles the internal SNN processing, primarily fetching synaptic data and managing events.

- **Initial/Wait State: S_IDLE (State 0)**

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

- **Transition out of Idle:**
  - To **S_FETCH_SYN** if a new neuron event arrives (CTRL_NEUR_EVENT).
  - To **S_READ_TREF** if a Time Reference event arrives (CTRL_NEUR_TREF).

- **Synapse Processing:**
  - **S_FETCH_SYN (State 1):** Fetches synaptic data. If the read length is zero, it returns to S_IDLE. Otherwise, it moves to wait for the output to be ready.
  - **S_WAIT_EVENT (State 2):** Waits until the output neuron is not busy (!OUT_NEUR_BUSY).
  - **S_WRITE_EVENT (State 3) S_WRITE_WAIT (State 4):** Writes the event. It loops back to S_FETCH_SYN if there are more post-synaptic events to process, or returns to S_IDLE when finished.

- **Time Reference Handling:**
  - **S_READ_TREF (State 5):** Reads the time reference. Waits until the output is not busy (!OUT_NEUR_BUSY).
  - **S_WRITE_TREF (State 6):** Writes the time reference. If the tref_counter reaches its limit, it returns to S_IDLE; otherwise, it loops back to S_READ_TREF to continue processing.

4.1.3 Olla Handshake FSM

This **second FSM**, specifically created for Olla communication, directly implements the state transitions of the 4-phase dual-rail protocol (although it appears to be a two-phase implementation in the provided snippet based on only VALID and NEUTRAL states).

- **Initial/Wait State: S_OLLA_IDLE (State 0)**
  - **Transition out of Idle:** Transitions to S_OLLA_VALID only when the main SNN FSM is in the S_WRITE_EVENT state **AND** the operation mode is set to Olla communication (OP_MODE == 1'b1).

- **Handshake Phases:**
  - **S_OLLA_VALID (State 1):** Asserts the Olla output validity/request. Transitions to S_OLLA_NEUTRAL upon receiving the **Acknowledge** from the Olla chip (OUTPUT_BITS_ONION_A_AO).
  - **S_OLLA_NEUTRAL (State 2):** De-asserts the Olla output. Transitions back to S_OLLA_IDLE only when the **Acknowledge** from Olla is de-asserted (!OUTPUT_BITS_ONION_A_AO), completing the handshake cycle.

4.2 Protocol Implementation and Data Structures

This subsection details the low-level logic governing the communication with the Olla chip and the utilization of internal memory resources for SNN operation and test sequencing.

4.2.1 Asynchronous 4-Phase Dual-Rail Protocol

The entire functional validation process relies on the robust implementation of the **4-Phase Dual-Rail Asynchronous Handshake Protocol**. This protocol is necessary to interface correctly with the Olla chip's clockless communication scheme.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

- **Implementation Location:** The protocol logic is primarily embedded within the **Neuron Core Second FSM** (Section 4.1.3), which manages the state transitions based on the Request (Req) and Acknowledge (Ack) signals.
- **Encoding:** Data bits are represented by two physical wires (**Dual-Rail**), where a logical '0' is represented by D_0= 1, D_1=0 and a logical '1' is represented by D_0=0, D_1= 1. The neutral state is D_0=0, D_1=0.
- **Phases of Handshake:**
    1. **Data Assertion (Request):** The transmitting side (FPGA or Olla) asserts the dual-rail encoded data and raises the Request signal (Req).
    2. **Data Acknowledge:** The receiving side (FPGA or Olla) captures the valid data and raises the Acknowledge signal (Ack).
    3. **Data De-assertion:** The transmitting side de-asserts both the data and the Request signal (Req).
    4. **Acknowledge De-assertion:** The receiving side lowers the Acknowledge signal (Ack), returning the channel to the neutral state, ready for the next transaction.
- **Role in Validation:** This protocol is used exclusively for both **injecting input stimuli** (from FPGA to Olla) and **capturing output spikes** (from Olla to FPGA).

4.2.2 Critical Memory Utilization

The FPGA utilizes dedicated Block RAM (BRAM) resources for critical data storage and SNN structure management.

Figure 7 Memory Structure

Memory Structure	***Location (Module)***	***Data Stored***	***Purpose in Design***
BRAM_Lut	Synaptic Core	**Cascaded Memory First Layer Data**	Stores the initial layer of synaptic data required by the Synaptic Core's modified logic. Its role change (Section 1.2.1) ensures the correct data alignment for the cascaded SNN.
BRAM_Synapse	Neuron Core	**Cascaded Memory Second Layer (Synaptic Data)**	Holds the actual synapse address and weight data. This BRAM is crucial for the second layer lookup based on the data provided by the first layer (BRAM_Lut).
BRAM_256x32_wrapper	Neuron Core	**Neuron State Variables**	Stores parameters such as Leaky-Integrate-and-Fire (LIF) thresholds and refractory periods for the neurons.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

The data stored within the BRAM_Synapse defines the routing mechanism for output events:

Bit Range	*Field Name*	*Description*
Bit 12 (MSB)	**OP_MODE**	**Critical Routing Control Bit:** This bit determines the destination of the processed spike event. **'1'** routes the event to the **Olla Chip Interface** (handshake logic); **'0'** routes the event to the **AER Output** logic. This information is loaded into the OP_MODE register within the Neuron Core FSM for handshake selection.
Bits 10:4	**Address Data**	The physical address or index within the subsequent layer or destination.
Bits 3:0	**Weight Data**	The synaptic weight value associated with the event.

5. Quick Start and Implementation Notes

This section provides critical, concise instructions necessary for the successful synthesis, deployment, and operation of the Olla Validation Platform design using the source code from the dedicated GitHub repository (last committed on October 22nd).

5.1 Preparation and Repository Setup

1. **Code Base:** Clone the design repository committed on October 22nd.
2. **In-System Debugging (ILA):** The code contains instantiation points for In-System Logic Analyzer (ILA) modules (e.g., for monitoring FSM states or data paths). **These ILA modules must be commented out** in the HDL source files before final synthesis and bitstream generation to avoid unnecessary resource consumption and potential timing issues.

5.2 Initialization and Memory (.mem) File Loading

The FPGA design relies on pre-initialized block RAM (BRAM) for the SNN structure. Three specific .mem files must be correctly formatted and loaded into the Vivado project.

A. bram_lut_data.mem (First Layer Base/Count)

This file defines the **base address and count** for the second memory layer (BRAM_Synapse).

- **Format:** Each line contains a **25-bit data word** represented in **Hexadecimal format**.
- **Data Structure:**
  - **Bits [24:8]:** Base Address information for the second layer memory. This defines the starting address to look up in the BRAM_Synapse.
  - **Bits [7:0]:** The number of synaptic entries for the second layer memory. This describes how many connections a pre-synaptic neuron has to post-synaptic neurons.
- **Purpose:** Provides the necessary lookup data for the pre-synaptic to post-synaptic neuron mapping required by the cascaded memory architecture.

B. bram_synapse_data.mem (Second Layer Synaptic Data)

This file contains the core synaptic address and weight data, including the critical routing control bit (as detailed in Section 4.2.2).

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/

- **Format:** Data must be written in **Hexadecimal format**, one entry per line, based on the following structure:
    - **Bit [11:11]: 1** for routing the event to the **Olla Chip Interface**; **0** for routing to the **AER Output**.
    - **Bits [10:4]:** Synapse Address data.
    - **Bits [3:0]:** Synapse Weight data.
- **Purpose:** Supplies the actual address and weight values used for SNN processing, critically determining whether an output spike is validated via Olla or routed internally via AER.

C. neuron_memory_data.mem (LIF Neuron Parameters)

This file contains the parameters required by the LIF neuron models (Section 4.2.2 and the uploaded image).

- **Format:** Data must align with the structure shown in the table below (based on original tinyODIN implementation).
- **Data Structure:**
    - Bits [11:0]: core (Membrane Potential State)
    - Bits [23:12]: thr (Firing Threshold Parameter)
    - Bits [30:24]: leak_str (Leakage Strength Parameter)
    - Bit [31:31]: neur_disable (Neuron Disable Parameter)
- **Purpose:** Initializes the operating parameters for the lif_neuron.v sub-module.

5.3 Pin Assignment and Constraints

The provided .xdc (XDC) file is crucial for mapping the design to the physical hardware.

1. **Initial XDC State:** The supplied .xdc file contains pin assignments specifically tailored for the **Kria KR260 Starter Kit**. These assignments utilize the **PMOD connectors** for external I/O communication.
2. **Olla Interface I/O:** The existing pin assignments for the PMOD connectors **can be reused** for the Olla chip communication, provided the custom Olla interposer board correctly interfaces with those PMOD headers. **No changes are required here** unless a different physical connector is used.
3. **Clock Pin Constraints (CRITICAL):** The clock pin assignments (CLK pins) in the .xdc file **must remain unchanged**. These pins are designed to be connected to the special clock-capable pins on the KR260 board, which are necessary for generating stable clock resources (MMCM/PLLs). Modifying these assignments will result in clock generation failure.
4. **Recommendation:** While the provided XDC is usable, it is strongly recommended that the design team **review and rewrite** the .xdc file to precisely match the final chosen hardware setup and signal integrity requirements.

© 2025 by Rijksuniversiteit Groningen (H. Basri Bilge). This work is licensed under CC BY 4.0. To view a copy of this license, visit https://creativecommons.org/licenses/by/4.0/